Secure Automotive Event Logging using ARM TrustZone and OP-TEE on STM32MP157

Aslam

Department of Computer Science and Engineering
Indian Institute of Technology Delhi

Under the guidance of Prof. Kolin Paul

July 2026

Abstract

The rapid growth of connected and autonomous vehicles has multiplied the number of Electronic Control Units (ECUs) that coordinate over the Controller Area Network (CAN) bus. Although CAN offers reliable communication, it provides no authentication, integrity, or confidentiality, exposing vehicles to replay, injection, tampering, and flooding attacks. Conventional logging compounds this weakness by recording events in ordinary Linux-managed memory, where a single host compromise allows an attacker to read keys and rewrite history. This work proposes a TrustZone-based secure logging framework on the STM32MP157. Every incoming CAN frame is handled first in the Secure World, where it is validated using CAN ID, timestamp, counter, and HMAC checks, together with replay and flood detection. Cryptographic keys are held in OP-TEE secure storage, inaccessible to Linux, and validated events are written to secure flash as a SHA-256 hash chain, yielding tamper-evident logs. The result is a hardware-isolated pipeline that markedly improves the forensic reliability and overall security of automotive ECUs.

Keywords: CAN Bus, TrustZone, OP-TEE, Secure Logging, STM32MP157.

1 Introduction

Modern vehicles contain many ECUs that communicate through the CAN bus. Although CAN provides reliable, deterministic communication, it does not provide **authentication**, **integrity**, or **confidentiality**. Because every node implicitly trusts every frame on the wire, an attacker with bus access can perform a range of attacks:

- **Replay Attack** — valid frames are captured and retransmitted.
- **Message Injection** — forged frames impersonate a legitimate ECU.
- **Tampering** — payload bits are altered so an ECU acts on false data.
- **Flooding** — high-priority frames saturate the bus and deny service.

To address these problems, this work proposes a TrustZone-based secure logging framework in which all CAN traffic is validated and recorded inside a hardware-isolated Secure World.

Objectives

- Validate every CAN packet before it influences vehicle behavior.
- Detect malicious packets through replay and flood analysis.
- Protect cryptographic keys from the Normal World.
- Store event logs securely in isolated flash.
- Prevent log tampering using cryptographic hash chaining.

Threat Model

The proposed framework assumes that an attacker has physical or logical access to the CAN bus and attempts to compromise the integrity of automotive communication. The attacker may:

- Replay previously captured CAN packets, inject forged CAN packets, or modify packet payloads during transmission.
- Flood the CAN bus with high-priority frames and attempt to compromise the Linux operating system.
- Attempt to access cryptographic keys stored in memory, and modify or delete stored event logs.

The following components are assumed to be **trusted**: ARM TrustZone hardware, the OP-TEE Secure OS, the Secure Boot process, and the underlying cryptographic algorithms (SHA-256 and HMAC).

Research Contributions

1. Secure interrupt based packet reception and TrustZone based packet validation.

2. Secure RAM isolation and OP-TEE protected key management.
3. Secure packet classification and Ground-Truth verification before Linux processing.
4. Hash-chain based tamper-evident logging in secure flash storage.

2 Existing System

In conventional automotive designs, CAN traffic flows directly into the rich OS with no hardware isolation. A received frame is handled by a standard Linux driver, copied into normal RAM, and optionally appended to a plaintext log in normal flash. The typical data path is shown in Figure 1.

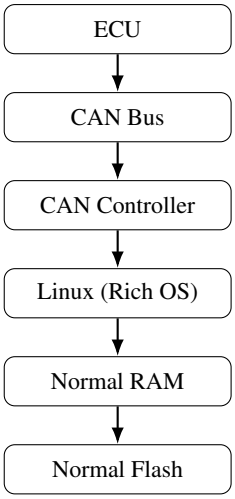


Figure 1: Existing CAN communication and logging data path (all in the Normal World).

Problems

Because all processing and storage occur in the Normal World, this architecture is exposed to:

- **Linux compromise** — a single host exploit exposes every buffer and key.
- **Key leakage** — keys held in normal memory can be extracted.
- **Replay** — no freshness check rejects retransmitted frames.
- **Fake ECU** — unauthenticated frames are accepted as genuine.
- **Log modification** — plaintext logs can be edited or deleted silently.

Research Gap

Existing works typically provide secure storage alone; they do not prove that a packet originates from a legitimate source, and they may miss packets due to reliance on a single core. Existing designs further assume the CAN controller itself is trusted.

Proposed Solution

This work closes that gap with a unified framework that validates every CAN frame and records it tamper-evidently, entirely within a hardware-isolated Secure World. Packet authentication, replay and flood detection, key custody, and hash-chained logging are combined into a single trusted pipeline on the STM32MP157, so neither a compromised Linux nor a DMA-capable peripheral can forge frames or alter the log.

3 Proposed Architecture

Secure Boot Process

Before Linux starts executing, the STM32MP157 follows a Secure Boot sequence to establish the hardware Root of Trust (Figure 2). Trusted Firmware-A verifies the integrity of OP-TEE before execution; OP-TEE then initializes Secure RAM, loads cryptographic keys from Secure Storage, and establishes the Secure World before Linux starts. As a result, Linux never gains direct access to secure memory or cryptographic keys.

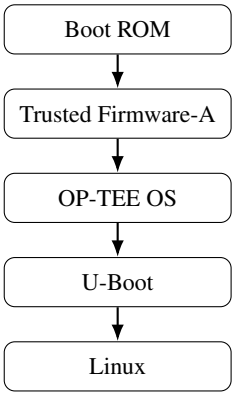


Figure 2: Secure Boot sequence establishing the hardware Root of Trust.

The proposed system inserts a TrustZone-backed pipeline between the CAN controller and the Normal World. Figure 3 shows the end-to-end architecture; each block is explained below.

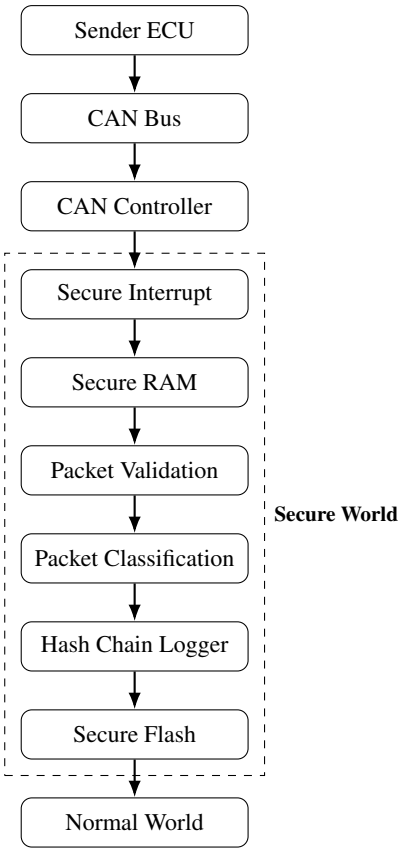


Figure 3: Proposed TrustZone-based secure packet-validation and logging pipeline.

CAN Controller. Receives the incoming CAN frame from the bus.
Secure Interrupt. Transfers control to the Secure World before any Normal World software observes the frame.
Secure RAM. The packet is copied into isolated memory that a compromised Linux cannot reach.
Packet Validation. Checks the **CAN ID**, **counter**, **timestamp**, and **HMAC**, and applies **replay** and **flooding** detection.
Classification. Validated frames are separated into **critical** (braking, steering, powertrain) and **normal** (comfort, telemetry) categories.
Secure Flash. Stores the hash-chained event log in a protected partition.

Packet Classification Policy

After successful validation, packets are classified by criticality. **Critical** packets — Brake System, Steering, Powertrain, and Airbag — remain inside the Secure World and are securely logged. **Normal** packets — Air Conditioning,

Window Control, Seat Adjustment, and Infotainment — are forwarded to the Normal World only after successful validation.

4 Secure Packet Validation

The complete validation flow proceeds through eight steps:

Step 1. The CAN controller receives a frame from the bus.
Step 2. A secure interrupt is generated.
Step 3. TrustZone switches execution into the Secure World.
Step 4. The packet is copied into Secure RAM.
Step 5. Validation is applied: CAN ID, timestamp, counter, HMAC, replay detection, and flood detection.
Step 6. The packet is classified as critical or normal.
Step 7. A hash is generated and chained to the previous log entry.
Step 8. The entry is stored into Secure Flash.

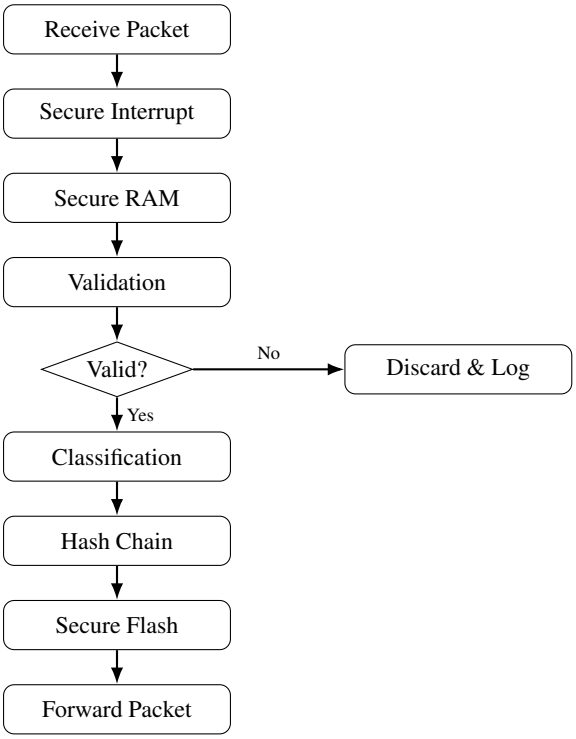


Figure 4: Secure packet-validation flowchart. Invalid frames are discarded and recorded rather than forwarded.

Ground Truth Verification

Although packets are validated inside the Secure World, an additional check is performed before Linux applications process the data: the Normal World compares packet information with trusted metadata, including the expected CAN ID, timestamp, counter value, stored HMAC, and the ECU database. Only packets that satisfy both Secure World validation *and* Normal World verification are delivered to user applications.

5 Secure Logging Mechanism

Hash Chain

Each log entry is linked to its predecessor by hashing the previous digest together with the new entry, forming an append-only chain (Figure 5).

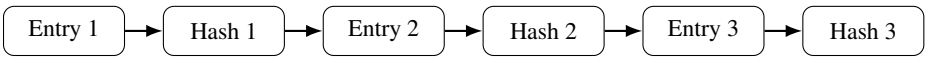


Figure 5: Hash chain linking successive log entries: each hash depends on the previous one.

The chain is defined by

$$H_i = \text{SHA256}\big(H_{i-1} \parallel \text{Entry}_i\big).$$

If an attacker changes **Entry 2**, then **Hash 2**, **Hash 3**, and every subsequent hash become invalid, so any modification to a past entry is immediately detectable.

Secure Key Storage

The HMAC key is stored inside **OP-TEE Secure Storage**, encrypted under a hardware-derived key. From the Normal World, Linux **cannot read**, **cannot modify**, and **cannot delete** it. Only the Trusted Application may access the key, and every cryptographic operation runs inside the Secure World so the plaintext key never leaves it.

Key Provisioning

The HMAC secret key is generated during device provisioning and securely stored inside OP-TEE Secure Storage before deployment; the same shared secret is provisioned into authorized ECUs during manufacturing. At runtime, Linux never accesses the secret key — only Trusted Applications inside OP-TEE can use it, all cryptographic operations execute entirely inside the Secure World, and plaintext keys never leave Secure RAM. This protects against key extraction even if the Normal World becomes compromised.

Memory Partition

DDR memory is partitioned so that assets never cross the security boundary (Figure 6).

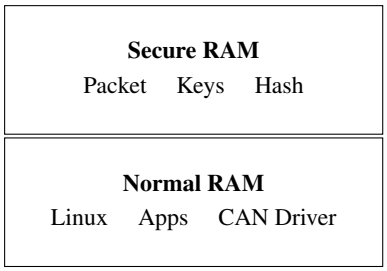


Figure 6: DDR partitioning between Secure RAM and Normal RAM.

6 Security Analysis

Table 1 maps each attack to the mechanism that defends against it.

Attack	**Countermeasure**
Replay	Counter + Timestamp
Injection	HMAC
Tampering	Hash Verification
Flooding	Rate Checking
Fake ECU	CAN ID + HMAC
DMA	Secure RAM
Log Modification	Hash Chain
Key Theft	OP-TEE Secure Storage

Table 1: Attacks and corresponding countermeasures.

Existing vs. Proposed Comparison

Feature	**Existing**	**Proposed**
Replay Protection	No	Yes
HMAC Authentication	No	Yes
Secure Key Storage	No	Yes
Hash Chain Logging	No	Yes
TrustZone Isolation	No	Yes
Secure RAM	No	Yes
Packet Classification	No	Yes
Ground Truth Verification	No	Yes

Table 2: Comparison of existing and proposed framework.

Performance Discussion

The proposed framework introduces a small computational overhead due to HMAC verification, SHA-256 hash generation, and packet validation. These operations, however, significantly improve security by preventing replay attacks, packet forgery, unauthorized memory access, and log tampering. Since security-critical automotive events occur less frequently than normal communication, the additional latency remains acceptable for practical automotive systems.

Advantages

- Secure packet validation before data reaches the rich OS.
- Secure, tamper-proof logging via hash chaining.
- Hardware isolation of buffers, keys, and logs.
- Secure key management inside OP-TEE, with Ground-Truth re-verification in the Normal World.

Current Limitations

- Hardware implementation is under development; cryptographic latency has not yet been measured.
- Secure interrupt configuration remains to be validated on STM32MP157 hardware.

Future Work

- Real hardware implementation on the STM32MP157, with CAN FD support for larger payloads.
- Hardware Security Module (HSM) integration and secure OTA updates anchored to the same TEE.
- Performance evaluation of the cryptographic overhead.

7 ECUs and the CAN Bus in Brief

An **Electronic Control Unit (ECU)** is a small embedded computer inside the vehicle—comprising a CPU, RAM, and a CAN controller—that reads sensors and drives actuators; a modern car contains dozens. These units share the **CAN bus**, a two-wire network (CAN_H / CAN_L) that carries differential signals. Because every ECU sees every frame on this shared medium, establishing trust is inherently difficult, which is precisely the problem the proposed framework addresses. Two complementary journeys describe the complete lifecycle of a CAN message: the **transmit path** (nine stages), in which a sensor reading becomes bits on the wire, and the **receive path** (twelve stages), in which a frame from the bus is validated and routed to the Secure or Normal World.

7.1 The Transmit Path: Sensor to CAN Bus

#	Stage	Action
1	Sensor	Physical parameter is sensed (e.g. temperature, speed, RPM); raw analog or digital signal generated.
2	ADC	Analog signal converted to digital data (only for analog sensors).
3	CPU Reads	CPU reads the digital value from the ADC or a digital interface.
4	RAM Stores	CPU stores the read sensor values in memory.
5	Create Frame	CPU builds the CAN frame carrying ID, DLC, Data, CRC, Counter, and Timestamp.
6	Write TX Buffer	Complete frame written to the controller's transmit buffer (memory-mapped registers).
7	Set TXREQ	CPU sets the Transmit Request bit (TXREQ = 1) in the CAN controller.
8	CAN Protocol	Controller performs arbitration, bit stuffing, CRC, ACK handling, and frame formatting in hardware.
9	CAN Bus	Frame transmitted on the differential CAN_H / CAN_L lines to all other ECUs.

Table 3: Transmit path: nine stages from sensor reading to bus.

At Stage 8 the controller handles every low-level CAN function in hardware: **arbitration** (highest-priority ID wins the bus), **bit stuffing** (keeps receivers in sync), **CRC** generation and checking (integrity), **ACK** handling (receipt

confirmed), and frame formatting. The result is driven onto CAN_H / CAN_L, where it is seen by every other ECU.

8 The Receive Path in Detail

Where the transmit path is a simple linear pipeline, the receive path is where security is enforced. Twelve stages carry a frame from the bus, through validation in the Secure World, to final storage. Stages 1–5 handle reception and isolation, Stage 6 performs validation, Stage 7 classifies, and Stages 8–12 route the packet to the appropriate world.

8.1 Reception: From Wire to Secure RAM

#	Stage	Action
1	CAN Transceiver	Physical layer: receives the differential signal (CAN_H, CAN_L) and converts it to a digital signal.
2	CAN Controller	Receives frame bits, checks CRC, checks ID / format, and stores the frame in the RX FIFO.
3	Interrupt Controller	Detects the new packet in the RX FIFO, generates an interrupt, and sends it to the Secure World (CPU).
4	Secure Handler	Secure interrupt received and acknowledged; the CPU reads the packet from the controller.
5	Copy to Secure RAM	Packet copied from the CAN controller into an isolated Secure RAM buffer.

Table 4: Receive path, stages 1–5: wire to Secure RAM.

8.2 Validation in the Secure World

Stage 6 subjects every packet to seven independent checks, all executed inside the Secure World. Only if all seven pass is the packet classified (Stage 7); otherwise it is dropped and the event is logged.

Check	Name	Purpose
6.1	CAN ID	Verify the CAN ID is in the allowed list.
6.2	HMAC	Verify HMAC for authenticity and integrity.
6.3	Flooding	Detect flooding (high frequency / abnormal rate).
6.4	Tamper	Check whether the packet was modified.
6.5	Timestamp	Verify the timestamp is within a valid window.
6.6	Counter	Verify the counter number is in sequence.
6.7	Replay	Detect a replayed packet (old timestamp / counter).

Table 5: The seven Secure-World validation checks (Stage 6). All must pass.

8.3 Classification and the Two Worlds

At Stage 7 the packet is classified by policy, rules, type, and priority, which decides its destination. Security-critical packets follow the **Secure World path** (8A); non-critical packets follow the **Normal World path** (8B):

- **8A** Secure World path — (1) hash chain: SHA256(PrevHash ‖ CurHash); (2) store packet and metadata in Secure Flash; (3) keep the record in secure storage with no Normal World acknowledgement.
- **8B** Normal World path — (1) write packet to shared memory and set a flag; (2) generate an interrupt to the Normal World; (3) the Normal World re-checks against the database, HMAC, timestamp, and counter (Stages 9–10); (4) valid packets are forwarded and invalid packets are dropped (Stage 11).

Finally, Stage 12 performs final storage: secure logs are written to Secure Flash, and normal logs to normal storage.

9 The Security Model at a Glance

The CAN bus trusts everyone by default, so every received packet must earn trust through layered, independent checks. The framework's guarantees can be grouped into six complementary properties:

- **Isolation.** Validation runs in the Secure World, in protected RAM the Normal World cannot reach.
- **Authenticity.** The HMAC proves the sender holds the key and the payload is unmodified.
- **Tamper-evidence.** A hash chain in Secure Flash makes any after-the-fact edit detectable.
- **Freshness.** Timestamp and counter together reject replayed or out-of-sequence packets.
- **Availability.** A flooding check catches abnormal traffic rates and denial-of-service attempts.
- **Second opinion.** The Normal World re-checks against ground truth before acting on data.

In summary, two journeys share one principle—trust must be earned. The transmit path (nine steps) digitizes a sensor reading, wraps it in a CAN frame, and drives it onto CAN_H / CAN_L. The receive path (twelve steps) isolates a packet in the Secure World, subjects it to seven checks, classifies it, and routes it to secure or normal handling, with logging either way.

Why STM32MP157?

STM32MP157 was selected because it provides the hardware security mechanisms required for trusted automotive systems: a dual ARM Cortex-A7 processor, ARM TrustZone support, OP-TEE compatibility, Secure Boot support, secure memory partitioning, a hardware cryptographic accelerator, and industrial-grade reliability. Together these features enable secure packet validation, protected key storage, and trusted event logging with minimal hardware modification.

Conclusion

This work proposed a TrustZone-based secure automotive logging framework for the STM32MP157. All received CAN packets are first handled in the Secure World, where the framework validates them using CAN ID, timestamp, counter, HMAC, replay detection, and flood detection before forwarding. Cryptographic keys remain protected inside OP-TEE secure storage, while validated events are stored in secure flash using a SHA-256 hash chain to provide tamper-evident logs. Compared with conventional Linux-based logging, this design substantially improves the security and forensic reliability of automotive ECUs.